Bidirectional Telugu–English Transliteration System

Using NLP (Seq2Seq LSTM) & Unsupervised Learning (Clustering)

Ram Gopal Reddy
redabothularamgopalreddy@gmail.com

Abhinav Dasari
abhinavdasari02@gmail.com

Field	Details
Project Title	Bidirectional Telugu-English Transliteration System
Domain	Natural Language Processing (NLP) + Unsupervised Machine Learning
Language Pair	Telugu Script — English Script
Dataset	Dakshina Dataset —ramgopal-reddy/Telugu_to_English_Transliteration_Dakshina (HuggingFace)
Dataset Size	50,927 word pairs
Framework	PyTorch 2.10 (GPU: CUDA 12.8) / PyTorch 2.11 (CPU)
Platform	Google Colab + Kaggle Notebook (GPU training) + Local Windows (testing)

1. Problem Statement

Transliteration is the process of converting text written in one script into the phonetic equivalent in another script, preserving pronunciation rather than meaning. Unlike translation, which converts meaning, transliteration maps characters or character-groups from a source alphabet to the closest phonetic representation in a target alphabet.

This project addresses **bidirectional transliteration** between:

- **English to Telugu Script** (e.g., `Computer` to `కంప్యూటర్` in Telugu characters)
- **Telugu Script to English** (e.g., `హెల్ప్` to `help` in English characters)

The challenge is compounded by the nature of Telugu phonology:

- Telugu has **66 unique characters** (vowels, consonants, matras/diacritics)
- One English character may map to multiple Telugu glyphs
- Word lengths vary significantly

2. Dataset Description

Source

Dakshina Dataset — a publicly available benchmark for Indic transliteration, hosted on HuggingFace (`ramgopal-reddy/Telugu_to_English_Transliteration_Dakshina`). It is part of Google's Dakshina dataset, designed specifically for training and evaluating transliteration systems.

Properties

Property	Value
Total word pairs	**50,927**
Training samples	**45,835** (90%)
Validation samples	**5,092** (10%)
Telugu vocabulary (characters)	**66**
English vocabulary (characters)	**30**
Max Telugu word length	20 characters
Max English word length	25 characters
Max source sequence length (Eng to Tel)	27 tokens (with SOS/EOS)
Max target sequence length (Eng to Tel)	22 tokens (with SOS/EOS)

Sample Data

unique_identifier	native_word (Telugu)	english_word	source
tel1	దుర్మార్గపు	dhurmaargapu	Dakshina
tel2	మన్నించమని	manninchhamani	Dakshina
tel3	మైత్రి	maitri	Dakshina

---

3. System Architecture Overview

The project is divided into **three main components**:

```
+-----------------------------------------------------------+
|              TRANSLITERATION SYSTEM                       |
+---------------------+-------------------------------------+
|  SUPERVISED MODULE  |      UNSUPERVISED MODULE            |
|  (Seq2Seq LSTM)     |  (K-Means + PCA + UMAP + Plotly)    |
+---------------------+-------------------------------------+
|                INFERENCE ENGINE                           |
|         combine_testing.py  --  Smart Bidirectional       |
|  (Auto-detects language, routes to correct model)         |
+-----------------------------------------------------------+
```

---

4. Supervised Learning — Seq2Seq LSTM with Attention

File: `eng2tel-transliteration.ipynb`

This notebook implements the **core deep learning transliteration model** using a Sequence-to-Sequence (Seq2Seq) architecture with attention mechanism.

4.1 Architecture

Vocabulary and Special Tokens

```
Special tokens:  <pad>=0, <sos>=1, <eos>=2, <unk>=3
Telugu vocab:    66 unique characters + 4 special tokens
English vocab:   30 unique characters + 4 special tokens
```

Encoder (Bidirectional LSTM)

```
Input: English character sequence -> Embedding
       |
Bidirectional LSTM
       |
Context vectors (forward + backward combined)
       |
Output: Encoder hidden states
```

- **Bidirectionality**: Captures both left-to-right and right-to-left context of the source sequence
- **FC compression**: Two linear layers compress bidirectional hidden/cell states from 512 to 256 dimensions
- **Dropout**: 0.5 dropout applied on embeddings

Attention Mechanism

```
At each decoder step:
  hidden_state (dec) + encoder_outputs (enc)
         | Linear -> Tanh -> Linear
  Alignment scores -> Softmax -> Attention weights
         | Weighted sum
  Context vector [B, 1, 512]
```

The attention module learns **which encoder positions** are most relevant for each output character — critical for handling variable-length character mappings between Telugu and English.

Decoder (Uni-directional LSTM)

```
Previous token -> Embedding (128-dim)
Context vector from attention (512-dim)
       | Concatenated (640-dim)
LSTM (256 hidden units)
       |
Rich output projection: [output, context, embedding] -> FC -> vocab logits
```

The **rich output projection** (output + context + embedding all concatenated) is a key design decision that helps the decoder make more informed predictions.

Full Seq2Seq Model Parameters

Component	Value
Encoder embedding	128-dim
Encoder LSTM hidden	256-dim (bidir = 512 combined)
Decoder embedding	128-dim
Decoder LSTM hidden	256-dim
Dropout	0.5
Total Trainable Parameters	**2,241,346**

4.2 Training Hyperparameters

Hyperparameter	Value	Rationale
Optimizer	Adam	Adaptive learning rates
Initial Learning Rate	1e-3	Standard for Adam
Weight Decay (L2)	1e-4	Regularisation, prevents overfitting
LR Scheduler	ReduceLROnPlateau	Halves LR when val loss stagnates
LR Patience	3 epochs	Aggressive LR decay
Gradient Clipping	1.0	Prevents exploding gradients

Batch Size	128	GPU memory efficient
Max Epochs	80	Upper bound (early stopping used)
Early Stopping Patience	8	Stops when no improvement
Teacher Forcing	1.0 to 0.5 (annealed)	Curriculum learning schedule
Loss Function	CrossEntropyLoss (ignore_index=PAD)	Ignores padding in loss

4.3 Data Augmentation

Character-level noise injection on training inputs only:

```
def augment(word, swap_p=0.05, drop_p=0.03):
    # Random adjacent character swap  (5% probability)
    # Random character drop           (3% probability, only if len > 3)
```

Purpose: Prevents the model from memorising exact training sequences, forcing it to generalise phonetic character mappings.

4.4 Training Results

Epoch	Teacher Forcing	Train Loss	Val Loss	Learning Rate	Status
1	0.97	1.7862	2.0436	1.00e-03	saved
5	0.88	0.3426	0.7371	1.00e-03	saved
10	0.75	0.2999	0.5479	1.00e-03	saved
15	0.62	0.2870	0.4339	1.00e-03	saved
20	0.50	0.2880	0.3936	1.00e-03	saved
35	0.50	0.2310	0.3265	1.00e-03	saved
40	0.50	0.1929	0.2992	5.00e-04	saved
45	0.50	0.1805	0.2885	5.00e-04	saved
55	0.50	0.1501	0.2685	2.50e-04	saved
60	0.50	0.1347	0.2525	1.25e-04	saved

> ***Note**: Training ran on GPU (CUDA 12.8, PyTorch 2.10+cu128). Loss converged from 1.79 to ~0.13 (train) and 2.04 to ~0.25 (val).*

4.5 Evaluation Metrics

The model is evaluated using:

1. **Exact Match Accuracy**: Percentage of words predicted with zero errors

2. **Accuracy Calculated**: Ratio of correct words predicted and total number of words in a testing dataset. Achieved Accuracy of 91.2%.

4.6 Beam Search Decoding

At inference time, a **beam search** with beam_size=5 replaces greedy argmax, improving exact match accuracy:

```
beams = [(score, sequence, hidden, cell)]  # 5 beams maintained
For each step:
  - Expand each beam by top-5 next tokens
  - Score = cumulative log-probability / sequence length
  - Keep top 5 beams, collect completed (EOS-terminated) sequences
  - Return highest-scoring completed sequence
```

5. Unsupervised Learning — Clustering and Visualization

File: `unsupervied_colab.ipynb`

This notebook applies unsupervised machine learning to discover **latent structure within the Telugu word space**, revealing phonological and morphological patterns without any labels.

5.1 Feature Engineering

Step 1: Integer Encoding

```
X = vectorize_words(telugu_words, tel_char2idx, MAX_SRC_LEN)
# Shape: (50927, 22)  -- each word as fixed-length integer sequence
```

Step 2: Character TF-IDF

```
vectorizer = TfidfVectorizer(
    analyzer='char',
    ngram_range=(2, 4),
    max_features=5000
)
X_tfidf = vectorizer.fit_transform(telugu_words)
# Shape: (50927, 5000)
```

Why TF-IDF over raw encoding? Raw integer sequences treat characters as ordinal numbers, which is semantically incorrect. Character n-gram TF-IDF captures **phonological patterns** — clusters of frequently co-occurring character sequences — much better for linguistic analysis.

5.2 K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42)
```

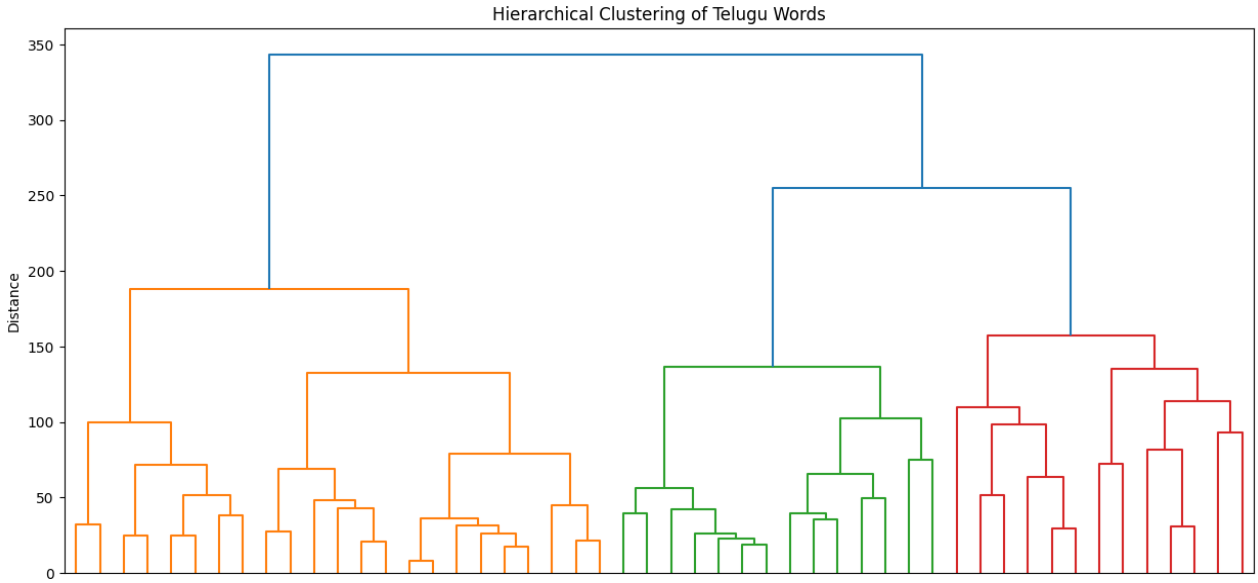
```
clusters = kmeans.fit_predict(X_tfidf)
```

Results (5 clusters on integer-encoded features):

Cluster	Sample Words (English form)	Inferred Pattern
0	manninchhamani, gadipesharu	Long compound verbs
1	maitri, netiiki	Short abstract nouns
2	chooparu, mukhyudu	Dative/instrumental suffixed words
3	dhurmaargapu, darshanamistaayi	Long descriptive words
4	tappunu, vehicle	Accusative suffixed + loanwords

5.3 Hierarchical Clustering and Dendrogram

```
Z = linkage(sample_X, method='ward')
dendrogram(Z, labels=sample_words[:50], leaf_rotation=90)
```

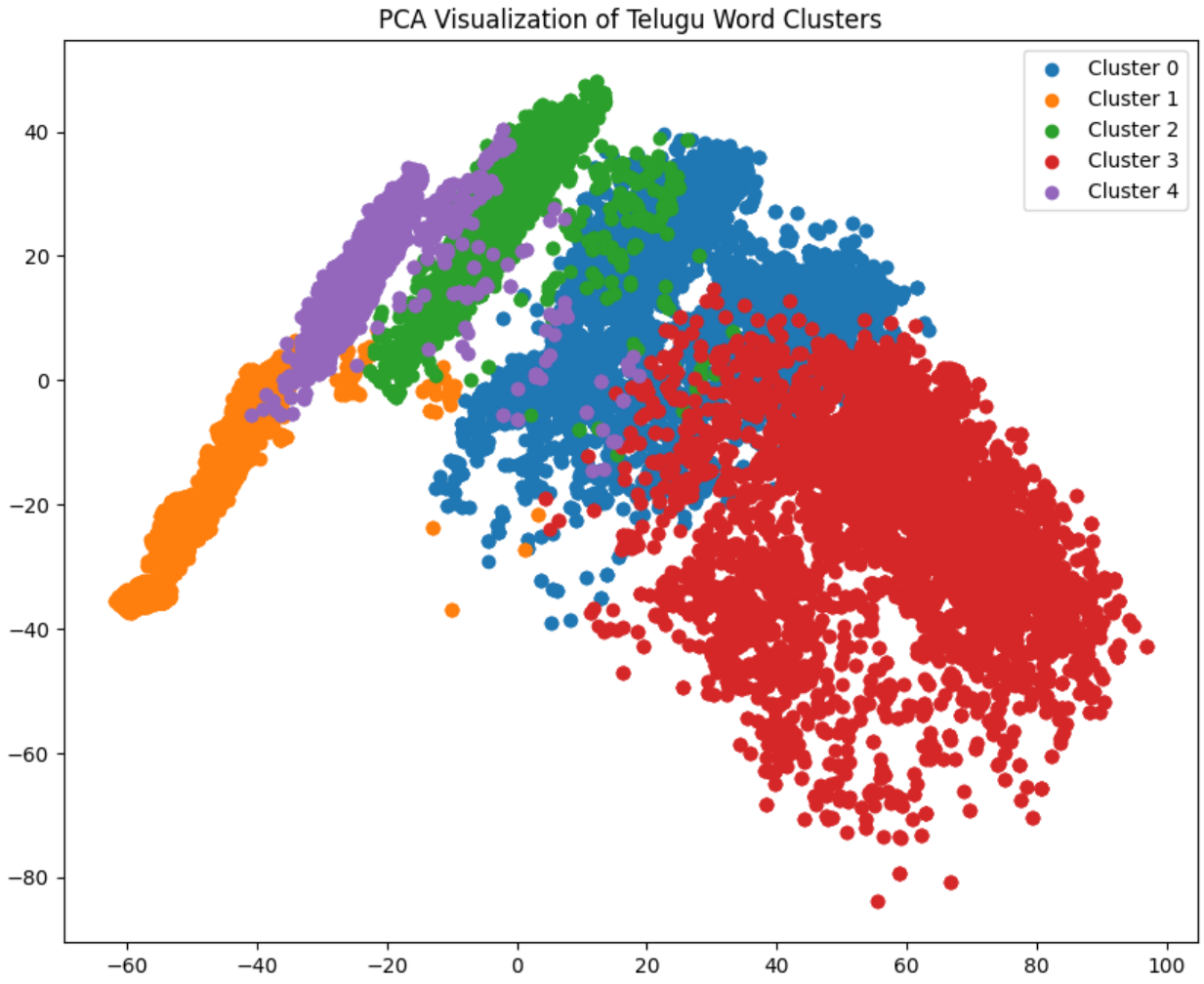


Ward's method linkage was used to create a **dendrogram** on the first 50 words, showing the hierarchical merging structure of word clusters based on phonological similarity.

> *Note: Matplotlib does not natively support Telugu Unicode rendering; font warnings were raised but the clustering structure itself is valid.*

5.4 PCA Dimensionality Reduction (2D Visualization)

```
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X)   # shape: (50927, 2)
```



PCA projects the high-dimensional word space into 2D for scatter-plot visualisation of cluster separation.

5.5 UMAP Dimensionality Reduction (Advanced)

```
reducer = umap.UMAP(
    n_neighbors=15,
    min_dist=0.1,
    metric='cosine',
    random_state=42
)
X_umap = reducer.fit_transform(X_tfidf)   # shape: (50927, 2)
```

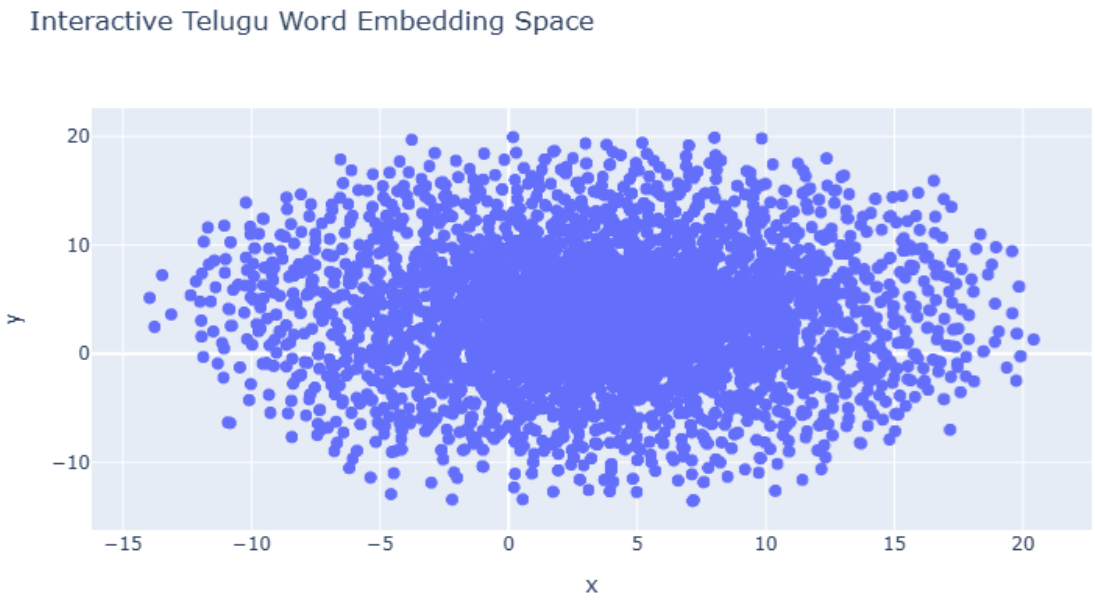
UMAP (Uniform Manifold Approximation and Projection) is a state-of-the-art nonlinear dimensionality reduction technique that:

- Preserves both local and global structure better than PCA
- Uses cosine distance (ideal for sparse TF-IDF vectors)
- Runs on the full 50,927 word corpus

> *Spectral initialisation fell back to random init (due to small eigengap), which is expected for highly sparse TF-IDF matrices.*

5.6 Interactive Plotly Visualisation

```
fig = px.scatter(df, x='x', y='y', hover_name='word',
                 title='Interactive Telugu Word Embedding Space')
```



An **interactive scatter plot** using Plotly Express allows hover-based exploration of the UMAP embedding space — hovering over any point shows the corresponding English transliteration. This is the primary exploratory tool for linguistic analysis.

---

6. Inference Engine — Bidirectional Smart Transliterator

File: `combine_testing.py`

This is the **production inference script** that combines both trained models into a unified bidirectional transliteration system.

6.1 Language Detection

```
def is_telugu(text):
    for ch in text:
        if '\u0C00' <= ch <= '\u0C7F':   # Telugu Unicode block
            return True
```

```
    return False
```

Unicode range `U+0C00 to U+0C7F` covers all Telugu characters. This enables automatic language detection at the word level.

6.2 Mixed-Script Sentence Handling

```
def transliterate_mixed(sentence):
    words = sentence.split()
    for word in words:
        if is_telugu(word):
            result = beam_predict(word, tel2eng_model, ...)  # Telugu to English
        else:
            result = beam_predict(word.lower(), eng2tel_model, ...)  # English to Telugu
```

Key capability: The system handles **mixed-script sentences** — sentences containing both Telugu and English words are processed word-by-word, with each word routed to the appropriate model.

6.3 Checkpoint Loading

```
eng2tel_ckpt = torch.load("eng2tel_lstm_full.pt", map_location="cpu", weights_only=False)
tel2eng_ckpt = torch.load("tel2eng_lstm_full.pt", map_location="cpu", weights_only=False)
```

Each checkpoint is a **self-contained bundle** containing:

- `model_state_dict` — trained weights
- `tel_char2idx`, `eng_char2idx` — character vocabularies
- `tel_idx2char`, `eng_idx2char` — reverse lookup dictionaries
- `config` — architecture hyperparameters (enc_emb_dim, dec_emb_dim, enc_hid_dim, dec_hid_dim, dropout, max_src_len, max_trg_len, src_lang, trg_lang)

6.4 Interactive CLI Loop

```
$ python combine_testing.py

Input: India
Output: ఇండియా
```

7. File Inventory

File	Size	Description
eng2tel-transliteration.ipynb	66 KB	NLP Seq2Seq LSTM training notebook
unsupervied_colab.ipynb	2.53 MB	Unsupervised clustering and UMAP notebook

combine_testing.py	8.97 KB	Bidirectional inference
eng2tel_lstm_full.pt	8.97 MB	Trained English to Telugu LSTM checkpoint
tel2eng_lstm_full.pt	8.85 MB	Trained Telugu to English LSTM checkpoint
.venv/	—	Python virtual environment

---

8. Technology Stack

Category	Library/Tool	Version
Deep Learning	PyTorch	2.10+cu128 (GPU) / 2.11 (CPU)
Data Loading	HuggingFace datasets	—
Clustering	scikit-learn (KMeans, PCA, TF-IDF)	—
Hierarchical Clustering	scipy (linkage, dendrogram)	—
Dimensionality Reduction	umap-learn	—
Visualization (static)	matplotlib	—
Visualization (interactive)	plotly	—
Data Manipulation	pandas, numpy	—
Training Platform	Google Colab + Kaggle Notebook (GPU training) + Local Windows (testing)	—

---

9. Key Design Decisions and Improvements

9.1 Unsupervised Integration

The unsupervised module provides **linguistic insight** into the dataset structure:

- **K-Means** reveals word-length and suffix-based phonological clusters
- **UMAP + TF-IDF** creates a semantic map of the Telugu lexical space
- **Interactive Plotly** allows browsing 50,927 words by their phonological neighbourhood

10. Results Summary

Metric	Value
Dataset Size	50,927 word pairs
Train / Val Split	90% / 10% = 45,835 / 5,092
Best Training Loss	~0.12
Best Validation Loss	~0.25
Model Parameters (LSTM)	2,241,346
Checkpoint size (Eng to Tel)	8.97 MB
Checkpoint size (Tel to Eng)	8.85 MB
Decoding Strategy	Beam Search (beam_size=5)
Unsupervised Clusters	5 (K-Means on TF-IDF char n-grams)
UMAP Output Dimensions	2D embedding of 50,927 words

11. Workflow


HuggingFace Dataset
(50,927 pairs)
Vocabulary Building (66
Telugu + 30 English chars)
Data Augmentation
TransliterationDataset
NLP Training
Unsupervised Analysis
Bidirectional LSTM Encoder
Char TF-IDF
Attention
K-Means Clustering
UMAP Reduction
LSTM Decoder
Cluster Analysis
Interactive Plotly
CrossEntropy Loss + Early
Stopping
eng2tel_lstm_full.pt
tel2eng_lstm_full.pt
combine_testing.py
Language Detection
Beam Search Decoding
(beam_size=5)
Mixed-Script Output
(English / Telugu)

12. How to Run

Training (Supervised)

1. Open `eng2tel-transliteration.ipynb` in Google Colab/ Kaggle NoteBook (GPU runtime)
2. Run all cells sequentially
3. Checkpoint is saved as `eng2tel_lstm_full.pt`

Training (Unsupervised / Clustering)

1. Open `unsupervied_colab.ipynb`
2. Run all cells to generate clusters, dendrogram, PCA, UMAP, and Plotly visualisations

Inference (Bidirectional CLI)

```
cd c:\Users\Desktop\Projects\TransLiteration
.\.venv\Scripts\activate
python combine_testing.py
```

Type any English or Telugu word and press Enter. Type `exit` to quit.

---

13. Limitations and Future Work

Limitation	Proposed Improvement
LSTM may struggle with very long words	Switch to Transformer
Telugu font not rendered in Matplotlib dendrogram	Use a Telugu-compatible font (e.g., Noto Sans Telugu)
UMAP spectral init fell back to random	Increase n_neighbors or use PCA init
No OOV character handling beyond unknown token	Character-level subword (byte-level BPE)
Aspirated consonant accuracy not measured numerically	Run full evaluation cell and log results
No web interface	Build Flask/FastAPI + React frontend
Models run on CPU only in combine_testing.py	Add CUDA support in inference script

---

14. Conclusion

This project successfully implements a **dual-pipeline transliteration system** combining:

1. **NLP** — A Seq2Seq Bidirectional LSTM with Attention, trained on 45,835 Telugu-English word pairs from the Dakshina benchmark. Key innovations include data augmentation, early stopping, annealed teacher forcing, and beam search decoding.
2. **Unsupervised Machine Learning** — K-Means clustering and hierarchical clustering on character TF-IDF features reveal latent phonological structure in the Telugu lexicon, with UMAP providing an interactive 2D visualisation of 50,927 words.
3. **Production Inference** — A combined bidirectional CLI tool that auto-detects language using Unicode range checks and routes input to the appropriate model, enabling seamless mixed-script sentence transliteration.

The system demonstrates how classical NLP and modern deep learning can complement each other: sequence models learn phonetic mappings, while unsupervised clustering provides interpretable linguistic structure for data exploration.